

# Concurrent Algorithms Assignment

## The Ball Tagging Game

### Objective

In this assignment, you will implement a simple **concurrent game simulation** involving three independent programs (or threads):

- **Code A** – Player A
- **Code B** – Player B
- **Code C** – Game Host

The purpose of this assignment is to practice **concurrent programming, synchronization, and shared resource management**.

---

### Game Description

A rectangular **court** is defined with a known:

- Length (L)
- Width (W)

Inside the court there are **n balls**, all initially colored **white**.

Two players compete to **touch balls and change their color**:

- **Player A (Code A):**  
When Player A touches a white ball, its color changes to **Red**.
- **Player B (Code B):**  
When Player B touches a white ball, its color changes to **Blue**.

The game runs for **2 minutes**. During this time, both players attempt to reach and color as many balls as possible.

At the end of the game, the **host program (Code C)** counts the number of **red balls** and **blue balls** and declares the winner.

---

### System Structure

You must implement **three concurrent components**:

#### 1. Code A – Player A

Responsibilities:

- Continuously attempt to select balls from the court.
  - If the selected ball is **white**, change its color to **Red**.
  - Ignore balls that are already **Red or Blue**.
  - Continue until the host signals that the game time has ended.
- 

## 2. Code B – Player B

Responsibilities:

- Continuously attempt to select balls from the court.
  - If the selected ball is **white**, change its color to **Blue**.
  - Ignore balls that are already **Red or Blue**.
  - Continue until the host signals that the game time has ended.
- 

## 3. Code C – Host Program

Responsibilities:

1. Initialize the game environment:
  - Define the court size (L, W)
  - Generate **n white balls** with random positions.
2. Start the players:
  - Launch **Code A** and **Code B** concurrently.
3. Maintain a **2-minute timer**.
4. When time expires:
  - Stop both players.
  - Count:
    - Number of **Red balls**
    - Number of **Blue balls**
5. Print the final result:

Example output:

Game Over

Red Balls : 42

Blue Balls : 37

Winner: Player A

If the counts are equal, print:

Result: Draw

---

## Concurrency Requirements

Your solution must demonstrate **proper concurrent programming techniques**.

You must address the following issue:

## Race Condition

Both players may try to **color the same ball at the same time**.

You must ensure that:

- A ball **cannot be colored twice**
- Only **one player succeeds** in coloring a white ball

Possible mechanisms include:

- Mutex / Locks
- Atomic operations
- Semaphores
- Other synchronization primitives

---

## Implementation Requirements

Students may use any of the following languages:

- C++
- Java
- Python

The implementation must include:

- Shared data structure representing the balls
- Concurrent execution of Player A and Player B
- Synchronization to avoid race conditions
- A timing mechanism for the 2-minute game duration

---

## Deliverables

Students must submit:

1. **Source Code**
  - Code A
  - Code B
  - Code C
2. **Short Report (1–2 pages)** including:
  - Description of the concurrency model used

- Synchronization mechanism used
- Explanation of how race conditions were prevented

---

## Bonus Challenge (Optional)

Add **movement simulation**:

- Each player moves randomly inside the court.
- A player can only color a ball if the **distance to the ball is below a threshold**.

---

## You may discuss the following questions

1. Speed of each player (equal, different, ..etc)
2. What **race condition** can occur if synchronization is not used?
3. What happens if both players try to color the **same ball simultaneously**?
4. How would the system behave if the **number of balls is very large**?
5. Optimal number of balls required to prevent draw.
6. **Bonus** - How can this game be implemented using **message passing instead of shared memory**?

---

**Bonus:** In the concurrent version, **Player A and Player B execute in parallel**.

1. Model the system using **two parallel processes**.
2. Explain how concurrency may improve the number of balls processed.
3. Let:

$$\begin{aligned}T_s &= \text{time of sequential execution} \\T_p &= \text{time of parallel execution}\end{aligned}$$

Define the **speedup**:

$$S = \frac{T_s}{T_p}$$

What is the **ideal speedup** for two players?

4. Why might the actual speedup be **less than the ideal value**?